



Informe de Aplicación Big Data

Análisis de Inspecciones Sanitarias de Restaurantes en Nueva York usando Dask, Modin y Spark

Integrantes	Participación
Astulle Zapata, David Alonso	100 %
Carlos Acosta, Rodrigo Dion	100 %
Corne Flores, Yofer Mauricio	100 %
Flores Berrio, Jorge Harlop	100 %
Silva Anampa, Manuel Jesus	100 %
Velarde Tipte, Jesús Gadiel	100 %

6 de mayo de 2026

Curso de Big Data

Docente: Lezama Benavides, Aldo Martin

1. Introducción

La seguridad alimentaria en establecimientos comerciales es un aspecto fundamental para la salud pública, especialmente en ciudades con alta densidad poblacional y gran actividad gastronómica como Nueva York. Los restaurantes son inspeccionados periódicamente por el Departamento de Salud de la ciudad con el fin de verificar el cumplimiento de normas sanitarias relacionadas con la manipulación de alimentos, higiene, control de plagas, condiciones del local y otros factores que pueden afectar directamente a los consumidores. Según el *New York City Department of Health and Mental Hygiene* (NYC Health, s. f.), estas inspecciones son no anunciadas, se realizan al menos una vez al año y las infracciones detectadas reciben puntajes que influyen en la calificación final del restaurante. En este sistema, un menor puntaje representa un mejor desempeño sanitario.

En este proyecto se analiza el dataset *DOHMH New York City Restaurant Inspection Results*, publicado en el portal de datos abiertos de la ciudad de Nueva York. Este conjunto de datos contiene registros de inspecciones sanitarias realizadas a restaurantes y cafeterías universitarias activas, incluyendo información sobre el establecimiento, fecha de inspección, tipo de cocina, infracciones detectadas, nivel de criticidad, puntaje obtenido, calificación asignada y ubicación geográfica (NYC OpenData, s. f.).

El objetivo principal del proyecto es construir un conjunto de KPIs sanitarios que permitan analizar el desempeño de los restaurantes de Nueva York a partir de sus inspecciones históricas. De manera específica, se busca identificar patrones de riesgo, evaluar la evolución de los establecimientos en el tiempo, detectar reincidencias en infracciones críticas y comparar el comportamiento sanitario según distrito, tipo de cocina y calificación obtenida. Con ello, el proyecto busca demostrar cómo el análisis de datos a gran escala puede apoyar la toma de decisiones en contextos de salud pública, fiscalización y gestión de establecimientos alimentarios.

2. Descripción del Dataset

2.1. Origen de los datos

El dataset utilizado en este proyecto es *DOHMH New York City Restaurant Inspection Results*, publicado en el portal *NYC OpenData* y proporcionado por el *Department of Health and Mental Hygiene* (DOHMH) de la ciudad de Nueva York. Este conjunto de datos pertenece a la categoría *Health* y reúne información relacionada con inspecciones sanitarias realizadas a establecimientos de comida, incluyendo restaurantes y cafeterías universitarias activas.

El conjunto de datos fue seleccionado porque presenta un contexto real de análisis para salud pública y gestión de establecimientos alimentarios. Además, permite construir indicadores sanitarios relacionados con riesgo, reincidencia, desempeño por distrito, evolución temporal de puntajes, tipos de cocina e infracciones críticas.

Para este proyecto, el dataset se trabajará en formato *CSV* y será almacenado en Google Cloud Storage para su posterior procesamiento mediante herramientas de Big Data.

2.2. Estructura del dataset

El dataset contiene registros asociados a inspecciones sanitarias realizadas a establecimientos de comida. Es importante señalar que no debe interpretarse como una lista

única de restaurantes, ya que un mismo establecimiento puede aparecer varias veces. Esto ocurre porque un restaurante puede tener inspecciones en distintas fechas y, además, una misma inspección puede registrar múltiples infracciones.

Por ello, la unidad de análisis original corresponde a un registro de inspección o citación sanitaria, mientras que para algunos indicadores será necesario agrupar los datos por restaurante, fecha de inspección, distrito, tipo de cocina o nivel de criticidad.

Cuadro 1: Resumen técnico del dataset

Elemento	Descripción
Nombre del dataset	<i>DOHMH New York City Restaurant Inspection Results</i>
Fuente	NYC OpenData
Entidad responsable	Department of Health and Mental Hygiene (DOHMH)
Categoría	Health
Formato utilizado	CSV
Cantidad aproximada de registros	Aproximadamente 297 mil filas
Cantidad de columnas	27 columnas
Unidad de registro	Registro asociado a una inspección o citación sanitaria
Frecuencia de actualización	Diaria
Fecha de corte utilizada	29 de abril de 2026

Una característica relevante del dataset es la existencia de registros con fecha de inspección 01/01/1900. Este valor no representa una inspección real, sino que indica que el establecimiento aún no cuenta con una inspección registrada. Por esta razón, dichos registros serán tratados durante la etapa de limpieza de datos.

Asimismo, algunas columnas pueden presentar valores nulos o categorías no válidas, como distritos marcados como **Missing**, puntajes ausentes o coordenadas geográficas incompletas. Estos casos serán considerados en la preparación de los datos antes de construir las consultas CRUD y los KPIs.

2.3. Variables principales

El dataset contiene 27 columnas. Para el análisis del proyecto, las variables más relevantes son aquellas que permiten identificar al restaurante, ubicarlo geográficamente, analizar sus inspecciones, evaluar las infracciones registradas y construir indicadores sanitarios.

Cuadro 2: Variables principales del dataset

Variable	Descripción
1. Identificación del establecimiento	
CAMIS	Identificador único del restaurante o establecimiento inspeccionado. Permite agrupar registros pertenecientes al mismo local.
DBA	Nombre comercial con el que opera públicamente el establecimiento.
PHONE	Número telefónico registrado del restaurante o administrador.
2. Ubicación del establecimiento	
BORO	Distrito donde se ubica el restaurante: Manhattan, Bronx, Brooklyn, Queens o Staten Island.
BUILDING	Número del edificio donde se encuentra el establecimiento.
STREET	Calle asociada a la dirección del establecimiento.
ZIPCODE	Código postal del establecimiento.
Latitude	Latitud geográfica del restaurante.
Longitude	Longitud geográfica del restaurante.
Location	Punto geográfico construido a partir de la latitud y longitud.
3. Información de la inspección	
INSPECTION DATE	Fecha en la que se realizó la inspección. El valor 01/01/1900 indica que el establecimiento aún no tiene inspección registrada.
ACTION	Resultado operativo de la inspección, por ejemplo, infracciones citadas, ausencia de infracciones, cierre o reapertura del establecimiento.
INSPECTION TYPE	Tipo de inspección realizada según el programa, proceso o motivo de evaluación.
RECORD DATE	Fecha en la que se generó o actualizó el registro dentro del dataset.
4. Infracciones sanitarias	
VIOLATION CODE	Código de la infracción detectada durante la inspección.
VIOLATION DESCRIPTION	Descripción textual de la infracción sanitaria registrada.
CRITICAL FLAG	Nivel de criticidad de la infracción: <i>Critical</i> , <i>Not Critical</i> o <i>Not Applicable</i> .

Variable	Descripción
5. Evaluación sanitaria	
SCORE	Puntaje sanitario asignado a la inspección. Un menor puntaje representa un mejor desempeño sanitario.
GRADE	Calificación asignada al restaurante, como A, B, C, N, Z o P.
GRADE DATE	Fecha en la que se emitió la calificación del restaurante.
6. Clasificación y variables urbanas	
CUISINE DESCRIPTION	Tipo de cocina declarado por el restaurante. Permite comparar riesgos sanitarios entre categorías gastronómicas.
Community Board	Junta comunitaria asociada a la ubicación del restaurante.
Council District	Distrito del consejo municipal correspondiente.
Census Tract	Zona censal donde se encuentra el establecimiento.
BIN	Identificador del edificio.
BBL	Identificador administrativo de borough, block y lot.
NTA	Área de tabulación vecinal asociada al restaurante.

3. Arquitectura Big Data en GCP

La arquitectura del proyecto se diseñó sobre Google Cloud Platform (GCP) bajo un enfoque de procesamiento *batch*. Esta decisión responde a la naturaleza del dataset, el cual se analiza como una fuente histórica de inspecciones sanitarias y no como un flujo de datos en tiempo real.

El pipeline integra tres elementos principales: Google Cloud Storage como capa de almacenamiento, Dataproc como entorno de cómputo distribuido y las herramientas Dask, Modin y Apache Spark para el procesamiento de datos. A partir de esta arquitectura se ejecutan tareas de limpieza, transformación, consultas CRUD y construcción de KPIs sanitarios.

3.1. Descripción general de la arquitectura

La arquitectura se organiza en cuatro capas: fuente de datos, ingesta batch, almacenamiento en la nube y procesamiento distribuido. Los resultados generados por el procesamiento se almacenan nuevamente en la nube, manteniendo trazabilidad entre los datos originales, los datos transformados y las salidas analíticas.

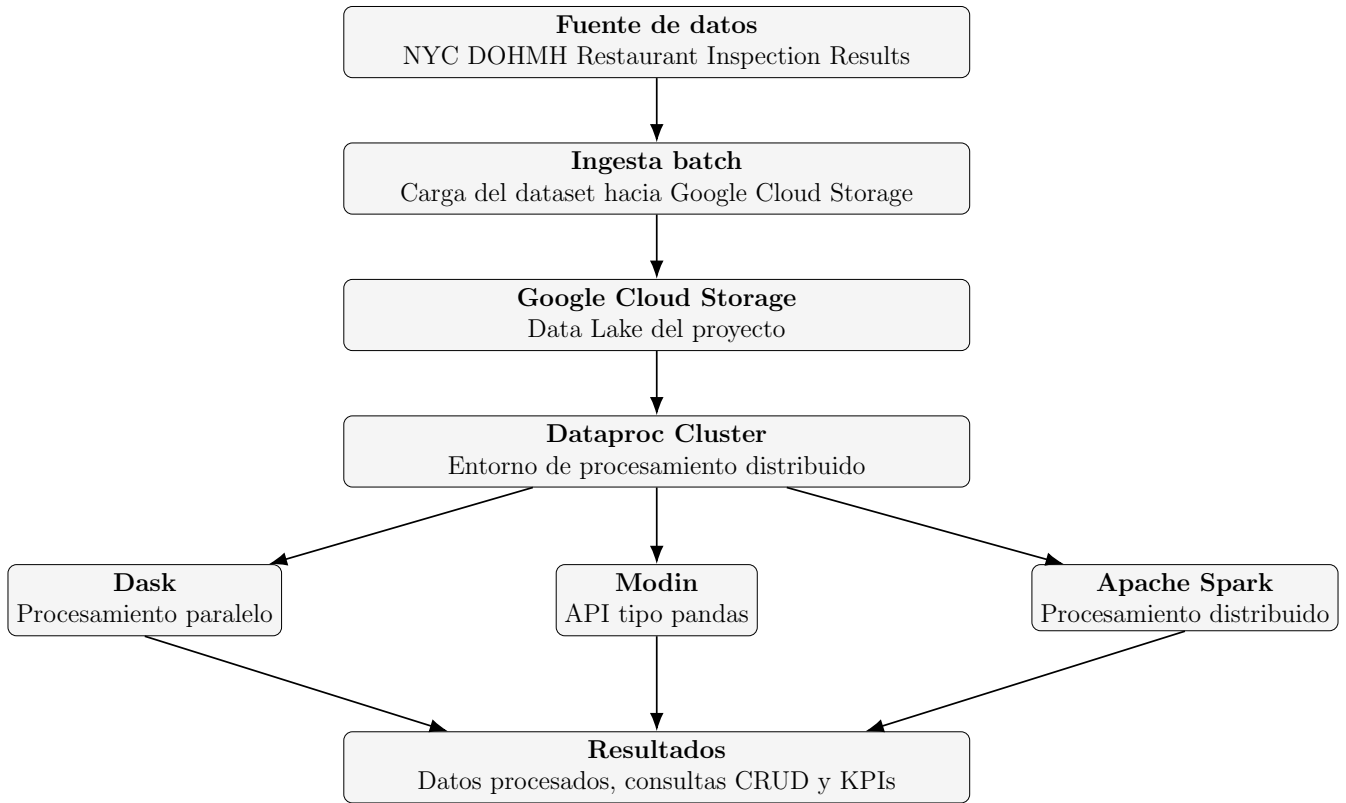


Figura 1: Arquitectura Big Data propuesta para el procesamiento del dataset de inspecciones sanitarias.

3.2. Flujo de datos del pipeline

El pipeline inicia con la obtención del dataset desde NYC OpenData. Luego, el archivo se carga en Google Cloud Storage, específicamente en la zona de datos crudos. Esta primera capa conserva el archivo original sin modificaciones para permitir reproducibilidad.

A continuación, el cluster de Dataproc lee los datos desde Google Cloud Storage y ejecuta los scripts de procesamiento. En esta etapa se realizan operaciones de limpieza, transformación, agregación y enriquecimiento de datos. Entre las tareas consideradas se encuentran la conversión de fechas, tratamiento de valores nulos, normalización de categorías, creación de variables auxiliares y generación de datasets derivados.

Finalmente, los resultados generados por Dask, Modin y Apache Spark se escriben nuevamente en Google Cloud Storage. Las salidas se organizan en datos procesados, resultados de consultas CRUD y tablas de KPIs, para luego ser utilizadas en el análisis final, el informe técnico y el video demostrativo.

3.3. Componentes de la arquitectura

La Tabla 3 resume los componentes principales de la arquitectura y su función dentro del pipeline.

Cuadro 3: Componentes principales de la arquitectura Big Data

Componente	Función	Rol en el proyecto
Dataset NYC Restaurants	Fuente de datos.	Proporciona registros de inspecciones sanitarias, infracciones, puntajes, calificaciones y ubicación de restaurantes.
Ingesta batch	Carga del archivo hacia la nube.	Permite procesar el dataset como una fuente histórica actualizable.
Google Cloud Storage	Almacenamiento central del pipeline.	Conserva datos crudos, datos procesados y resultados analíticos.
Data Lake	Organización lógica del almacenamiento.	Separa las zonas raw/ , processed/ y results/ .
Dataprocc	Entorno de cómputo distribuido.	Ejecuta los scripts de procesamiento sobre un cluster administrado.
Nodo maestro	Coordinación del cluster.	Gestiona la ejecución de trabajos y distribución de tareas.
Nodos trabajadores	Ejecución paralela.	Procesan particiones de datos y operaciones de transformación/agregación.
Apache Spark	Motor de procesamiento distribuido.	Se usa para consultas, agregaciones y KPIs sobre grandes volúmenes de datos.
Dask	Procesamiento paralelo en Python.	Se usa para operaciones tabulares mediante particiones.
Modin	Procesamiento paralelo con API tipo pandas.	Se usa para acelerar operaciones familiares de análisis tabular.
Resultados analíticos	Salidas del pipeline.	Incluyen datos procesados, consultas CRUD y KPIs sanitarios.

3.4. Justificación técnica

La arquitectura propuesta cumple con los requerimientos del proyecto porque integra almacenamiento en la nube, procesamiento distribuido y generación de indicadores analíticos. Google Cloud Storage se utiliza como Data Lake porque permite centralizar archivos de entrada, salidas intermedias y resultados finales en una estructura organizada.

El enfoque batch es adecuado debido a que el análisis se realiza sobre un dataset histórico. Esto evita incorporar componentes de transmisión en tiempo real que no son necesarios para los objetivos del proyecto, reduciendo complejidad y enfocando el trabajo en limpieza, transformación, agregaciones y KPIs.

Dataprocc se selecciona como entorno principal porque permite ejecutar trabajos distribuidos en GCP mediante un cluster administrado. Esta configuración se alinea con el enfoque del curso, ya que permite trabajar con un nodo maestro y nodos trabajadores sin configurar manualmente toda la infraestructura.

Finalmente, el uso de Apache Spark, Dask y Modin permite comparar diferentes enfoques de procesamiento. Spark representa el enfoque distribuido clásico para Big Data; Dask permite escalar operaciones en Python mediante particiones; y Modin ofrece una alternativa paralela compatible con operaciones tipo pandas. En conjunto, estas herramientas permiten cubrir las etapas de procesamiento distribuido, CRUD y construcción de KPIs solicitadas en el proyecto.

4. Implementación en GCP

La implementación del proyecto se realizó en Google Cloud Platform (GCP), utilizando Google Cloud Storage como capa de almacenamiento y Dataprocc como entorno de procesamiento distribuido. Google Cloud Storage permite centralizar el dataset original, los datos procesados y los resultados analíticos, mientras que Dataprocc proporciona el cluster necesario para ejecutar procesos de limpieza, transformación, consultas CRUD y KPIs mediante Dask, Modin y Apache Spark.

En esta sección se describen las configuraciones realizadas en GCP: creación del bucket, organización del Data Lake, configuración del cluster de Dataprocc y evidencias de implementación.

4.1. Configuración del bucket en Google Cloud Storage

Para implementar la capa de almacenamiento en la nube, se creó un bucket en Google Cloud Storage. Este bucket funciona como repositorio central del proyecto y permite que el entorno de procesamiento acceda a los datos desde una ubicación común.

El bucket creado para el proyecto se denomina:

`proyectobigdata2026`

La configuración utilizada se resume en la Tabla 4.

Cuadro 4: Configuración del bucket en Google Cloud Storage

Elemento	Configuración utilizada
Nombre del bucket	proyectobigdata2026
Servicio utilizado	Google Cloud Storage
Tipo de ubicación	Región
Ubicación	us-central1 (Iowa)
Clase de almacenamiento	Standard
Espacio de nombres jerárquico	Inhabilitado
Rapid Cache	Inhabilitado
Prevención de acceso público	Activada
Control de acceso	Uniforme
Política de eliminación no definitiva	Activada con duración predeterminada de 7 días
Control de versiones de objetos	Inhabilitado
Política de retención	Inhabilitada
Tipo de encriptación	Administrada por Google

Se seleccionó la región **us-central1** para mantener coherencia con la región del cluster de Dataproc y evitar transferencias innecesarias entre regiones. No se utilizó una configuración multi-región porque el proyecto no requiere replicación geográfica ni alta disponibilidad distribuida entre regiones.

La clase de almacenamiento elegida fue *Standard*, debido a que el dataset será consultado con frecuencia durante las etapas de limpieza, transformación, consultas CRUD y generación de KPIs. Las clases *Nearline*, *Coldline* y *Archive* no fueron seleccionadas porque están orientadas a datos de acceso poco frecuente.

En cuanto a seguridad, se activó la prevención de acceso público para evitar la exposición de los archivos en Internet. Además, se utilizó control de acceso uniforme, lo que permite administrar permisos a nivel de bucket mediante IAM. Finalmente, se mantuvo activa la política de eliminación no definitiva por 7 días, permitiendo recuperar objetos eliminados accidentalmente durante el desarrollo. No se activó el versionado de objetos ni una política de retención, ya que el proyecto no requiere conservar múltiples versiones históricas ni bloquear la eliminación de archivos.

4.2. Organización del Data Lake

El bucket fue organizado siguiendo una estructura de Data Lake por zonas. Esta organización permite separar los datos originales, los datos procesados y los resultados analíticos del proyecto.

La estructura base creada en el bucket fue la siguiente:

```
gs://proyectobigdata2026/
```

```
|-- raw/  
|-- processed/  
|-- results/
```

Sobre esta base, se define la siguiente organización lógica para el desarrollo completo del pipeline:

```
gs://proyectobigdata2026/
```

```
|-- raw/  
| |-- restaurant_inspections.csv  
|  
|-- processed/  
| |-- dask/  
| |-- modin/  
| |-- spark/  
|  
|-- results/  
| |-- kpis/  
| | |-- dask/  
| | |-- modin/  
| | |-- spark/  
|  
| |-- outputs/  
| | |-- dask/  
| | |-- modin/  
| | |-- spark/  
|  
|-- notebooks//  
| |-- dask_pipeline.py  
| |-- modin_pipeline.py  
| |-- spark_pipeline.py
```

La carpeta **raw/** almacena el dataset original descargado desde NYC OpenData. Esta zona conserva los datos sin modificaciones, permitiendo reproducir el procesamiento desde la fuente inicial.

La carpeta **processed/** almacena los datos limpios y transformados. Dentro de ella se separan las salidas por herramienta de procesamiento: Dask, Modin y Apache Spark. Esta separación permite identificar qué tecnología generó cada resultado.

La carpeta **results/** almacena las salidas analíticas finales. En **results/kpis/** se guardan las tablas de indicadores sanitarios, mientras que en **results/outputs/** se almacenan resultados derivados de consultas CRUD, agregaciones y tablas intermedias.

La carpeta **notebooks/** se considera para almacenar el código fuente final del proyecto. Aunque durante el desarrollo se utiliza Jupyter en Dataproc para probar y ejecutar código

de manera interactiva, los scripts permiten entregar una versión ordenada y reproducible del procesamiento.

Cuadro 5: Organización del Data Lake en Google Cloud Storage

Directorio	Contenido	Propósito
raw/	Dataset original en formato CSV.	Conservar una copia base sin modificaciones.
processed/dask/	Datos procesados con Dask.	Separar las salidas generadas mediante procesamiento paralelo con Dask.
processed/modin/	Datos procesados con Modin.	Almacenar las salidas generadas con operaciones tipo pandas aceleradas.
processed/spark/	Datos procesados con Apache Spark.	Guardar transformaciones y agregaciones generadas mediante Spark.
results/kpis/	Tablas finales de indicadores.	Centralizar los KPIs sanitarios construidos durante el proyecto.
results/outputs/	Resultados de consultas CRUD y agregaciones.	Almacenar salidas analíticas complementarias.
notebooks/	Scripts Python del proyecto.	Respaldar el código fuente final de manera ordenada.

Esta organización permite mantener trazabilidad entre el archivo original, las transformaciones aplicadas y los resultados obtenidos. Además, facilita la revisión del proyecto, ya que las salidas quedan separadas según su etapa y herramienta de procesamiento.

4.3. Configuración del cluster Dataproc

Para la ejecución del procesamiento distribuido se creó un cluster de Dataproc en Google Cloud Platform. Este cluster fue utilizado como entorno principal para ejecutar notebooks en JupyterLab y procesar el dataset almacenado en Google Cloud Storage mediante Dask, Modin y Apache Spark.

La configuración final del cluster fue definida considerando tres criterios: disponer de una arquitectura distribuida real, mantener recursos suficientes para el procesamiento tabular del dataset y controlar el costo del proyecto. La Tabla 6 resume la configuración utilizada.

Cuadro 6: Configuración general del cluster Dataproc

Elemento	Configuración utilizada
Servicio utilizado	Dataproc
Nombre del cluster	proyectobigdata-2026-2
Región	us-west1
Zona	us-west1-a
Tipo de cluster	Estándar
Arquitectura	1 nodo administrador y 2 nodos trabajadores
Trabajadores primarios	2
Trabajadores secundarios	0
Imagen de Dataproc	2.3.29-debian12
Componente adicional	Jupyter habilitado
Acceso HTTP a interfaces web	Habilitado
Red VPC	default
Modo de IP	Solo IP interna
Bucket de configuración	proyectobigdata2026
Bucket temporal	Bucket temporal administrado por Dataproc en us-west1
Estado del cluster	RUNNING

Se utilizó un cluster estándar porque esta opción permite representar una arquitectura distribuida real. En esta configuración, el nodo administrador coordina la ejecución de los trabajos y los nodos trabajadores ejecutan tareas en paralelo. Esta estructura resulta adecuada para el proyecto, ya que permite trabajar con Spark como motor distribuido principal y, además, ejecutar notebooks interactivos para validar los procesos con Dask y Modin.

Aunque el bucket principal del proyecto es **proyectobigdata2026**, el cluster fue creado en la región **us-west1**. Esta configuración funcionó correctamente para el desarrollo, ya que el cluster pudo leer y escribir datos en Google Cloud Storage. Sin embargo, para un entorno productivo sería recomendable ubicar el bucket y el cluster en la misma región, con el fin de reducir latencia, evitar transferencias innecesarias entre regiones y mantener una arquitectura más optimizada.

El cluster fue creado con IP interna únicamente. Esta decisión mejora la seguridad porque los nodos no quedan expuestos directamente mediante direcciones IP públicas. Sin embargo, durante la instalación de dependencias externas desde JupyterLab se identificó que era necesario habilitar salida a internet. Por ello, se configuró Cloud NAT como

complemento de red, permitiendo que los nodos accedan a repositorios externos como PyPI sin asignarles IP pública.

Configuración de nodos

La configuración de recursos del nodo administrador y los nodos trabajadores se muestra en la Tabla 7.

Cuadro 7: Configuración de nodos del cluster Dataproc

Tipo de nodo	Máquina	Recursos configurados
Nodo administrador	n4-standard-4	1 nodo administrador con 100 GB de disco Hyperdisk Balanced.
Nodo trabajador	n4-standard-2	2 nodos trabajadores, cada uno con 200 GB de disco Hyperdisk Balanced.

El nodo administrador utiliza una máquina **n4-standard-4**, mientras que cada nodo trabajador utiliza una máquina **n4-standard-2**. Esta configuración permite separar la coordinación del cluster de la ejecución de tareas distribuidas. El nodo administrador gestiona servicios como YARN, Spark History Server, HDFS NameNode y JupyterLab, mientras que los nodos trabajadores aportan los recursos de cómputo para la ejecución de procesos distribuidos.

El almacenamiento principal del proyecto se mantuvo en Google Cloud Storage, por lo que los discos del cluster fueron utilizados principalmente para archivos temporales, dependencias, caché de ejecución y operaciones intermedias. No se configuraron GPU ni SSD locales, ya que el proyecto se centra en procesamiento tabular y no en entrenamiento de modelos o cargas intensivas de cómputo gráfico.

Configuración de acceso, seguridad y red

En la configuración de acceso del cluster se habilitó el alcance de plataforma de nube, lo que permitió la comunicación con servicios de GCP, especialmente Google Cloud Storage. Esto fue necesario para leer el dataset desde **raw/**, guardar bases procesadas en **processed/** y almacenar resultados en **results/**.

Cuadro 8: Configuración de acceso, seguridad y red del cluster

Elemento	Configuración utilizada
Modo de IP	Solo IP interna
Red VPC	default
Alcance de plataforma de nube	Habilitado
Permiso de almacenamiento	Lectura y escritura sobre Cloud Storage
Encriptación	Clave administrada por Google
Cloud KMS	No habilitado
Confidential Computing	No habilitado
Autenticación personal del cluster	No habilitada
Cloud NAT	Configurado posteriormente para salida a internet

El uso de IP interna permitió mantener una configuración más segura, ya que los nodos del cluster no quedaron expuestos directamente hacia internet. No obstante, esta configuración impidió inicialmente instalar paquetes externos desde JupyterLab. La creación de Cloud NAT resolvió este problema, permitiendo instalar dependencias como Dask, Modin, Ray, PyArrow y librerías auxiliares desde el propio entorno del cluster.

No se habilitó Cloud KMS ni Confidential Computing porque el proyecto no requiere administración personalizada de llaves ni procesamiento confidencial avanzado. La encriptación administrada por Google fue suficiente para el alcance académico del trabajo.

Uso de JupyterLab y ejecución de notebooks

El componente Jupyter fue habilitado durante la creación del cluster. Esto permitió acceder a JupyterLab directamente desde Dataproc y ejecutar notebooks dentro del entorno del cluster. Esta decisión fue importante porque permitió validar de forma interactiva la lectura desde Google Cloud Storage, la limpieza de datos, las consultas CRUD y los KPIs.

El flujo de trabajo utilizado fue el siguiente:

1. Cargar el dataset desde `gs://proyectobigdata2026/raw/`.
2. Ejecutar notebooks en JupyterLab sobre el cluster de Dataproc.
3. Validar la limpieza, transformaciones, consultas CRUD y KPIs con Dask, Modin y Spark.
4. Guardar las bases limpias en `processed/`.
5. Guardar los resultados analíticos en `results/outputs/` y `results/kpis/`.

6. Organizar la lógica final en scripts Python para respaldar la entrega del código fuente.

JupyterLab se utilizó como entorno principal de desarrollo y validación, mientras que los scripts Python representan la versión ordenada y reproducible del procesamiento. Esta separación permitió trabajar de forma exploratoria durante el desarrollo y, al mismo tiempo, conservar una estructura adecuada para la entrega final del proyecto.

Interfaces de monitoreo

El cluster habilitó interfaces web asociadas a Dataproc, incluyendo JupyterLab, YARN ResourceManager, Spark History Server, HDFS NameNode y MapReduce Job History. Estas interfaces permitieron revisar el estado del entorno, monitorear ejecuciones y validar el comportamiento de los trabajos distribuidos.

En particular, JupyterLab fue utilizado para ejecutar los notebooks del proyecto, mientras que Spark History Server y YARN ResourceManager sirvieron como herramientas de apoyo para verificar la ejecución de trabajos Spark dentro del cluster.

4.4. Evidencias de implementación

A continuación, se presentan las evidencias visuales de la implementación realizada en Google Cloud Platform. Estas capturas respaldan la creación del bucket, la configuración del almacenamiento, la organización del Data Lake y la creación del cluster de Dataproc.

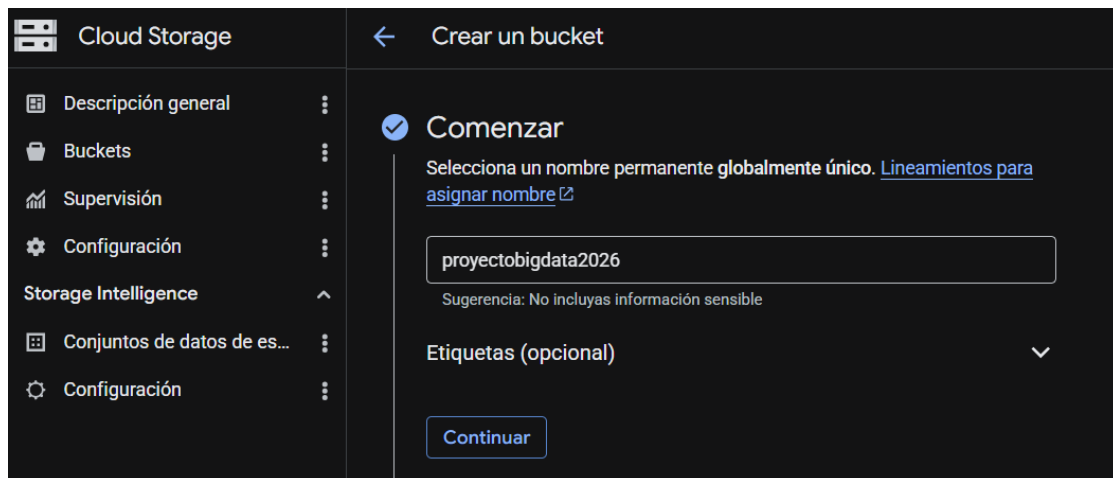


Figura 2: Definición del nombre del bucket `proyectobigdata2026`.

✓ **Elige dónde almacenar tus datos**

Esta opción define la ubicación geográfica de tus datos y afecta el costo, el rendimiento y la disponibilidad. No se puede cambiar más adelante. [Más información](#)

Tipo de ubicación

- ☐ **Multi-region**
Máxima disponibilidad en el área más amplia
- ☐ **Dual-region**
Alta disponibilidad y baja latencia en 2 regiones
- ☒ **Region**
Latencia mínima dentro de una sola región

us-central1 (Iowa) ▼

☐ **Agregar replicación entre buckets a través del Servicio de transferencia de almacenamiento**
A medida que se agregan o modifican datos, repícalos en otro bucket, lo que te permitirá almacenar una copia que sigue diferentes parámetros de configuración de buckets, por ejemplo, región, clase de almacenamiento, etcétera. [Más información](#)

☐ **Zone**
Latencia mínima dentro de una sola zona

Continuar

Figura 3: Selección de ubicación regional `us-central1` para el bucket.

• **Elige cómo almacenar tus datos**

Una clase de almacenamiento determina los costos del almacenamiento, la recuperación y las operaciones con diferencias mínimas en el tiempo de actividad. Elige si deseas administrar los objetos de forma automática o especifica una clase de almacenamiento predeterminada según la duración con la que planeas almacenar tus datos y tu carga de trabajo o caso de uso. [Learn more](#)

- ☐ **Autoclass** ⓘ
Realiza la transición automáticamente de cada objeto a las clases Estándar o Nearline según la actividad a nivel de objeto para optimizar el costo y la latencia. Se recomienda si la frecuencia de uso puede ser impredecible. Se puede cambiar a una clase predeterminada en cualquier momento. [Detalles de precios](#)
- ☒ **Configurar una clase predeterminada**
Se aplica a todos los objetos del bucket, a menos que modifiques de forma manual la clase por objeto o establezcas reglas de ciclo de vida de los objetos. Se recomienda cuando el uso es muy predecible.
 - ☐ **Rapid**
Ideal para operaciones de lectura y escritura de mayor frecuencia. Solo disponible para zonas.
 - ☒ **Standard**
La mejor opción para el almacenamiento a corto plazo y los datos de acceso frecuente
 - ☐ **Nearline**
Ideal para copias de seguridad y datos a los que se accede menos de una vez al mes
 - ☐ **Coldline**
Ideal para recuperación ante desastres y datos a los que se accede menos de una vez por trimestre
 - ☐ **Archive**
La mejor opción para la conservación digital a largo plazo de los datos a los que se accede menos de una vez al año

Figura 4: Selección de clase de almacenamiento *Standard*.

Elige cómo almacenar tus datos

Clase de almacenamiento predeterminada: Standard

Espacio de nombres jerárquico: Inhabilitado

Rapid Cache: Inhabilitado

- ### Elige cómo controlar el acceso a los objetos

Impedir el acceso público

Restringe el acceso público a los datos a través de Internet. Esto evitará que el bucket se use para el hosting web. [Más información](#)

☒
Aplicar la prevención de acceso público a este bucket

Control de acceso

☒
Uniforme

Garantiza el acceso uniforme a todos los objetos del bucket mediante el uso exclusivo de permisos a nivel de bucket (IAM). Esta opción se aplicará de manera permanente después de 90 días. [Más información](#)

Tu organización requiere buckets para usar la opción de acceso uniforme a nivel de bucket.

☐
Preciso

Especifica el acceso a objetos individuales mediante el uso de permisos a nivel de objeto (LCA) además de los permisos a nivel de bucket (IAM). [Más información](#)

Continuar

Figura 5: Configuración de prevención de acceso público y control de acceso uniforme.

- ### Elige cómo proteger los datos de objeto

Tus datos siempre están protegidos con Cloud Storage, pero también puedes elegir entre estas opciones adicionales de protección de datos para agregar capas de seguridad extras.

Protección de datos

☒
Política de eliminación no definitiva (para la recuperación de datos)

Cuando se habilita esta opción, el bucket y sus objetos se conservan durante un período específico después de que se borran, y pueden restablecerse durante este tiempo. [Más información](#)

☒
Usar la duración de retención predeterminada

Todos los buckets tienen una duración de eliminación no definitiva de 7 días de forma predeterminada, a menos que el administrador de tu organización haya personalizado este valor.

☐
Establecer duración de retención personalizada

Especifica por cuánto tiempo se deben conservar este bucket y sus objetos después de que se borren. Si estableces una duración de "0", se inhabilitará la eliminación no definitiva, lo que significa que se borrarán de forma permanente todos los objetos borrados.

☐
Control de versiones de objetos (para el control de versión)

Para restablecer objetos borrados o reemplazados. Para minimizar el costo de almacenamiento de versiones, te recomendamos limitar la cantidad de versiones no actuales por objeto y programarlas para que venzan después de una cantidad de días. [Más información](#)

☐
Retención (para el cumplimiento)

Para impedir que se borren o modifiquen los objetos del bucket durante un período específico.

Encriptación de datos

Figura 6: Configuración de protección de datos del bucket.

My First Project

Buscar (/) recursos, documentos, productos y más

Buscar

Buckets

Crear

Actualizar

Ir a ruta

Filtro

Filtrar depósitos

Nombre ↓

Fecha de creación

Tipo de ubicación

Ubicación

Clase de almacenamiento predeterminada ?

projectobigdata2026

24 abr 2026, 7:16:38 p.m.

Region

us-central1

Standard

Figura 7: Bucket projectobigdata2026 creado en Google Cloud Storage.

projectobigdata2026						
Ubicación	Clase de almacenamiento	Acceso público	Protección			
us-central1 (Iowa)	Standard	No público	Borrar de forma no definitiva			
Objetos	Configuración	Permisos	Protección	Ciclo de vida	Observabilidad	Informes de inventario Operaciones
Depósitos > projectobigdata2026						
Crear carpeta Subir Transferir los datos Crear operaciones por lotes Nuevo Otros servicios						
Filtrar solo por prefijo de nombre		Filtro Filtrar objetos y carpetas				Mostrar Solo o
	Nombre	Tamaño	Tipo	Fecha de creación ?	Clase de almacenamiento	Última modificación
	.ipynb_checkpoints/	—	Carpeta	—	—	—
	google-cloud-dataproc-metainfo/	—	Carpeta	—	—	—
	notebooks/	—	Carpeta	—	—	—
	processed/	—	Carpeta	—	—	—
	raw/	—	Carpeta	—	—	—
	results/	—	Carpeta	—	—	—

Figura 8: Creación de carpetas principales del Data Lake: raw/, processed/ y results/.

Define tu clúster

A cluster is a fully managed, scalable environment designed specifically for processing data using Apache Spark and other open source frameworks. Choose a region close to your primary data source for best results.

Nombre del clúster *
 proyectobigdata-2026-2

Región *
 us-west1

Zona *
 Cualquiera

Tipo de clúster
 Estándar (1 principal, N trabajadores)

Trabajadores primarios: 2, trabajadores secundarios: 0

☐ Habilitar Lightning Engine
 Activa esta opción para acelerar tus trabajos de Spark con Lightning Engine [Learn more](#)

Versión

Versión de la imagen y fecha de lanzamiento
 2.3-debian12 (Lanzamiento de la primera versión: 9/6/2025)

Figura 9: Definición general del cluster de Dataproc: nombre, región, tipo de cluster y número de trabajadores.

Serie
 N4

Con la tecnología de la plataforma de CPU Emerald Rapids de Intel

Tipo de máquina
 n4-standard-4 (4 CPU virtuales, 2 núcleos, 16 GB de memoria)

	vCPU	Memory
	4	16 GB

Plataforma de CPU y GPU

Tamaño del disco principal *
 100 GB

Primary disk type *
 Hyperdisk Balanced ...

IOPS IOPS

Throughput MB/s

Cantidad de SSD locales *
 0 x 375GB

Interfaz de SSD local
 SCSI

Figura 10: Configuración de recursos del nodo administrador.

Serie **N4** ▼
 Con la tecnología de la plataforma de CPU Emerald Rapids de Intel

Tipo de máquina **n4-standard-2 (2 CPU virtuales, 1 núcleos, 8 GB de memoria)** ▼



vCPU	Memory
2	8 GB

▼ Plataforma de CPU y GPU

Number of worker nodes * **2** ?

Tamaño del disco principal * **100** GB ?

Primary disk type * **Hyperdisk Balanced ...** ▼ ?

IOPS IOPS ?

Throughput MB/s ?

Cantidad de SSD locales * **0** x 375GB ?

Interfaz de SSD local **SCSI** ▼ ?

Figura 11: Configuración de recursos de los nodos trabajados.

Conectores de formato de tabla abierta

☐ Iceberg ? ☐ Delta ? ☐ Hudi ?

Muestra más componentes ^

☐ Docker ?

☐ Trino ?

☐ Flink ?

☒ Jupyter Notebook ?

☐ Hive WebHCat ?

☐ Ranger ?

☐ ZooKeeper ?

☐ Solr ?

☐ Zeppelin Notebook ?

☐ Jupyter Kernel Gateway ?

☐ Pig ?

☒ Habilitar las UIs de componentes

Figura 12: Uso de Jupyter en Dataproc para la ejecución interactiva del procesamiento.



Figura 13: Configuración de detención programada y seleccion del bucket GCS

Clústeres

Create cluster

Actualizar

Iniciar

Detener

Borrar

Regiones

+ 5 alertas recomendadas

Filtro

Busca el clúster por propiedades y presiona Intro

El servidor no pudo completar tu solicitud.

☐	Nombre ↓	Estado	Región	Zona	Versión de la imagen base	Motor	Total de nodos trabajadores
☐	proyectobigdata-2026-2	En ejecución	us-west1	us-west1-a	2.3.29-debian12	Default	2

Figura 14: Creación completa del cluster.

5. Procesamiento Distribuido

En esta sección se describen las operaciones concretas de limpieza y transformación aplicadas al dataset, así como el uso específico de cada herramienta dentro del pipeline. Las definiciones técnicas de Dask, Modin y Apache Spark, así como su rol dentro de la arquitectura, fueron presentadas en la Sección 3.

5.1. Procesamiento con Dask

Dask fue utilizado como una herramienta de procesamiento paralelo basada en *DataFrames* particionados. A diferencia de pandas, Dask no ejecuta inmediatamente todas las operaciones, sino que construye un grafo de tareas que se materializa cuando se invoca una acción como `compute()` o una escritura de resultados. Esta característica permite trabajar con operaciones sobre datos de mayor tamaño sin cargar todo el dataset de forma inmediata en memoria.

En el proyecto, Dask se empleó para realizar limpieza, transformación, creación de variables auxiliares, consultas CRUD y construcción de KPIs sanitarios. El flujo de trabajo se organizó en tres etapas: lectura y limpieza desde la capa `raw/`, almacenamiento de un checkpoint limpio en `processed/dask/`, y posterior ejecución de consultas y KPIs desde la base procesada.

Lectura del dataset desde Google Cloud Storage

El dataset fue leído directamente desde Google Cloud Storage, utilizando como entrada el archivo almacenado en la zona de datos crudos:

```
GCS_RAW_PATH = "gs://proyectobigdata2026/raw/restaurant_inspections.csv"

df_raw = dd.read_csv(
    GCS_RAW_PATH,
    dtype=dtype_config,
    assume_missing=True,
    blocksize="32MB",
    storage_options={"token": "google_default"}
)
```

Se definió explícitamente el tipo de dato de las columnas para evitar errores de inferencia durante la lectura distribuida. Además, se utilizó un tamaño de bloque controlado para generar particiones manejables durante la ejecución en Jupyter dentro del cluster de Dataproc.

Preparación y limpieza de datos

La limpieza inicial incluyó la normalización de campos textuales, conversión de fechas, conversión del puntaje sanitario a formato numérico y eliminación lógica de registros no válidos para el análisis. También se excluyeron registros con fecha de inspección 1900-01-01, debido a que este valor representa establecimientos sin inspección registrada.

Entre las principales transformaciones aplicadas se encuentran:

- Estandarización de campos textuales como `dba`, `boro`, `grade` y `critical_flag`.
- Conversión de `inspection_date`, `grade_date` y `record_date` a formato fecha.
- Conversión de `score` a valor numérico.
- Creación de variables temporales como `year`, `month` y `season`.
- Creación de variables sanitarias como `is_critical`, `has_violation`, `risk_level` y `sanitary_status`.
- Construcción de `inspection_id` para representar una inspección a partir de restaurante, fecha y tipo de inspección.

El siguiente fragmento resume parte de la creación de variables auxiliares:

```
df_clean["year"] = df_clean["inspection_date"].dt.year
df_clean["month"] = df_clean["inspection_date"].dt.month
```

```

df_clean["is_critical"] = (
    df_clean["critical_flag"]
    .str.lower()
    .eq("critical")
    .astype("int64")
)

df_clean["has_violation"] = (
    df_clean["violation_code"].notnull().astype("int64")
)

df_clean["inspection_id"] = (
    df_clean["camis"].astype(str) + "_" +
    df_clean["inspection_date"].dt.strftime("%Y-%m-%d") + "_" +
    df_clean["inspection_type"].astype(str)
)

```

Clasificación de infracciones y grupos de inspección

Además de la limpieza general, se crearon variables derivadas para facilitar el análisis sanitario. La variable `violation_category` agrupa códigos de infracción en categorías analíticas como contaminación de alimentos, plagas, origen no aprobado, permisos y documentación, sin violación y otros. Asimismo, la variable `inspection_group` agrupa los tipos de inspección en categorías como *Initial*, *Re-inspection*, *Compliance*, *Reopening*, *Limited* y *Other*.

Estas variables permitieron simplificar el análisis posterior, especialmente para las consultas CRUD y los KPIs asociados a tipos de violación y reincidencia.

Corrección de distritos usando código postal

Durante la limpieza se identificaron valores inválidos en la variable `boro`, como 0, Missing o valores nulos. Para corregir estos casos, se construyó una tabla auxiliar que relaciona `zipcode` con `boro` a partir de registros válidos. Luego, esta relación se usó para completar valores inválidos cuando existía correspondencia con el código postal.

El resultado de esta transformación se almacenó en una nueva columna llamada `boro_corrected`, evitando modificar directamente el campo original.

```

zip_boro_pdf = (
    df_clean[
        df_clean["boro"].notnull() &
        (~df_clean["boro"].isin(["0", "Missing", "MISSING", "missing"])) &
        df_clean["zipcode"].notnull()
    ][["zipcode", "boro"]]
    .drop_duplicates()
    .compute(scheduler="single-threaded")
)

```

Checkpoint en formato Parquet

Una consideración importante en Dask es que los *DataFrames* representan un grafo de tareas y no necesariamente una tabla materializada en memoria. Por ello, después de la limpieza y transformación se generó un checkpoint en formato Parquet dentro de Google Cloud Storage.

Este checkpoint permitió materializar la base limpia y evitar que las consultas CRUD y los KPIs recalcularan toda la limpieza desde el archivo CSV original. La ruta utilizada fue:

`gs://proyectobigdata2026/processed/dask/restaurant_inspections_clean_parts/`

Debido a las limitaciones de memoria del entorno interactivo, la escritura se realizó por particiones. Cada partición fue calculada, almacenada temporalmente en el nodo y luego subida a Google Cloud Storage. Esta estrategia permitió reducir el uso de memoria y mantener control sobre el avance del proceso.

```
for i in range(df_analysis.npartitions):
    pdf_part = df_analysis.get_partition(i).compute(
        scheduler="single-threaded"
    )

    local_file = f"/tmp/dask_processed_parts/part-{i:04d}.parquet"

    pdf_part.to_parquet(
        local_file,
        index=False,
        engine="pyarrow"
    )

    subprocess.run(
        f"gcloud storage cp {local_file} {GCS_PROCESSED_DASK}",
        shell=True,
        check=True
    )
```

Luego de este proceso, la base limpia fue leída nuevamente desde Parquet:

```
df_analysis = dd.read_parquet(
    "gs://proyectobigdata2026/processed/dask/restaurant_inspections_clean_parts/*.*parquet",
    storage_options={"token": "google_default"}
)
```

A partir de este punto, las consultas CRUD y los KPIs se ejecutaron sobre la base procesada, reduciendo el costo de recomputación.

Uso del cliente Dask

Luego de materializar la base limpia en Parquet, se activó un cliente local de Dask para apoyar la ejecución de consultas y agregaciones. Se utilizó una configuración moderada para evitar saturar la memoria del entorno Jupyter.


```
cluster = LocalCluster(
    n_workers=1,
    threads_per_worker=2,
    processes=False,
    memory_limit="5GB"
)

client = Client(cluster)
```

Esta configuración permitió aprovechar el modelo de ejecución paralelo de Dask en operaciones como agrupaciones, conteos, filtros y generación de resultados agregados, sin forzar la carga completa del dataset en memoria.

Consultas CRUD con Dask

Las consultas CRUD implementadas con Dask se resumen en la Tabla 9. Estas operaciones se ejecutaron sobre la base procesada almacenada previamente en formato Parquet.

Cuadro 9: Consultas CRUD implementadas con Dask

Código	Tipo	Descripción
DASK-C1	Create	Creación de la variable <code>violation_category</code> , que agrupa los códigos de infracción en categorías sanitarias.
DASK-C2	Create	Creación de la variable <code>inspection_group</code> , que agrupa los tipos de inspección en categorías generales.
DASK-R1	Read	Cálculo del puntaje sanitario promedio por año de inspección.
DASK-R2	Read	Cálculo de la distribución porcentual de calificaciones sanitarias.
DASK-U1	Update	Enriquecimiento de la variable de distrito mediante <code>boro_corrected</code> , usando el código postal como referencia.
DASK-D1	Delete lógico	Construcción de una base válida para análisis mediante filtrado de registros sin distrito válido, sin puntaje o sin fecha de inspección.

Aunque se desarrollaron seis operaciones, se consideran como parte del conjunto oficial las necesarias para cubrir generación de variables, lectura de agregados, enriquecimiento y eliminación lógica.

KPIs calculados con Dask

Dask fue utilizado para construir indicadores orientados a analizar reincidencia sanitaria, composición de infracciones y tiempos de mejora de los restaurantes. Estos KPIs

aprovechan operaciones distribuidas como filtros, agrupaciones, uniones y cálculos sobre fechas, ejecutadas sobre la base limpia previamente almacenada en formato Parquet.

Cuadro 10: KPIs calculados con Dask

KPI	Nombre	Propósito
KPI 3	Reincidencia post-clausura	Identificar restaurantes que presentan infracciones o nuevas inspecciones después de haber registrado una clausura por parte del DOHMH.
KPI 4	Tasa de violaciones por tipo	Determinar qué categorías de infracciones concentran mayor proporción dentro del total de violaciones sanitarias.
KPI 7	Tiempo promedio de subsanación	Medir el tiempo que tarda un restaurante en pasar de una calificación desfavorable (B o C) a una calificación favorable (A).

KPI 3: Reincidencia post-clausura Definición. Este KPI identifica restaurantes que, después de registrar una clausura, vuelven a presentar inspecciones posteriores con infracciones. Para ello, primero se obtiene la primera fecha de clausura por restaurante y luego se analizan los registros posteriores a dicha fecha.

Fórmula.

$$\text{Violaciones post-clausura}_i = \sum_{j=1}^n 1(\text{fecha inspección}_{ij} > \text{fecha clausura}_i)$$

Donde i representa al restaurante y j representa las inspecciones posteriores a la clausura.

La Tabla 11 muestra los restaurantes con mayor cantidad de violaciones registradas después de una clausura.

Cuadro 11: Top 10 restaurantes con mayor reincidencia post-clausura

CAMIS	Restaurante	Distrito	ZIP	Violaciones	Inspecciones
50138270	MEEM SPICY GROCERY AND DELI	Manhattan	10016	67	11
50111296	BIG WONG	Manhattan	10013	55	10
41696014	NOISETTE	Queens	11103	52	9
50009411	DELICIAS CALENAS #4	Queens	11435	50	8
50130108	CENTRAL PARK CAFE	Manhattan	10019	46	8
41628099	LLOYD'S CARROT CAKE	Manhattan	10029	44	10
50130345	AFFY'S GRILL	Queens	11378	43	7
50124605	CHEF TAN	Manhattan	10003	43	6
50139259	SHANGHAI TIME	Manhattan	10018	41	6
50048617	HALAL BUFFET	Queens	11435	41	10

El restaurante con mayor reincidencia post-clausura fue *MEEM SPICY GROCERY AND DELI*, con 67 violaciones distribuidas en 11 inspecciones posteriores a la clausura. También se observa una concentración importante de casos en Manhattan y Queens. Este indicador permite identificar establecimientos que requieren seguimiento adicional, ya que continuaron presentando infracciones después de un evento crítico como la clausura.

KPI 4: Tasa de violaciones por tipo **Definición.** Este KPI mide la proporción de violaciones sanitarias agrupadas por categoría. Para ello, los códigos de infracción fueron clasificados en grupos como contaminación de alimentos, plagas, permisos y documentación, origen no aprobado y otros.

Fórmula.

$$\text{Tasa de violaciones}_c = \frac{\text{Violaciones de la categoría } c}{\text{Total de violaciones registradas}} \times 100$$

Donde c representa cada categoría de violación.

La Tabla 12 muestra la distribución porcentual de las violaciones por tipo.

Cuadro 12: Tasa de violaciones por tipo

Categoría de violación	Porcentaje
Otros	64.34 %
Contaminación de alimentos	19.39 %
Plagas	12.48 %
Permisos y documentación	2.92 %
Origen no aprobado	0.86 %

La categoría *Otros* concentra la mayor proporción de violaciones, con 64.34 %. Esto indica que una parte importante de las infracciones no pertenece a los grupos principales definidos inicialmente y podría requerir una clasificación más detallada en trabajos futuros. Entre las categorías específicas, las más relevantes son contaminación de alimentos, con 19.39 %, y plagas, con 12.48 %. Estos resultados sugieren que los principales focos de riesgo sanitario están relacionados con condiciones de manipulación, conservación o contaminación de alimentos y con la presencia de plagas.

KPI 7: Tiempo promedio de subsanación **Definición.** Este KPI mide el tiempo que tarda un restaurante en mejorar su condición sanitaria después de recibir una calificación desfavorable. Para ello, se identifican inspecciones con calificación B o C y se busca la primera inspección posterior en la que el restaurante obtiene calificación A.

Fórmula.

$$\text{Días de subsanación}_i = \text{mín}(\text{fecha A posterior}_i) - \text{fecha de calificación desfavorable}_i$$

El promedio por restaurante se calcula como:

$$\text{Tiempo promedio de subsanación}_i = \frac{\sum_{k=1}^m \text{Días de subsanación}_{ik}}{m}$$

Donde i representa al restaurante y k cada evento de mejora identificado.
La Tabla 13 muestra los restaurantes con mayor tiempo promedio de subsanación.

Cuadro 13: Top 10 restaurantes con mayor tiempo promedio de subsanación

CAMIS	Restaurante	Días promedio
41619055	SONS OF ESSEX	1091.0
50061281	JOE & PAT'S	1085.0
40385885	HUNTER'S STEAK & ALE HOUSE	1078.0
50098711	PHO BEST III	1071.0
41418789	THE ORIGINAL EMILIO'S PIZZA	1071.0
40379149	MONTE'S TRATTORIA	1065.0
50086215	LARUSTICA PIZZA	1064.0
41621699	MI PASO CENTRO AMERICANO RESTAURANT	1060.0
50098945	DIOS PROVEEDOR BAKERY	1058.0
41476400	CHOP CHOP	1058.0

Los resultados muestran que algunos restaurantes presentan tiempos de subsanación superiores a los 1000 días, lo cual indica una demora considerable entre una calificación desfavorable y una mejora posterior a calificación A. El caso más alto corresponde a *SONS OF ESSEX*, con 1091 días promedio. Este indicador permite identificar establecimientos donde las mejoras sanitarias toman más tiempo, lo que puede ser útil para priorizar seguimiento, fiscalización o revisión de reincidencias.

Almacenamiento de resultados

Los resultados generados por Dask fueron almacenados nuevamente en Google Cloud Storage. Los datos procesados se guardaron en la carpeta `processed/dask/`, mientras que las salidas de consultas CRUD y KPIs se almacenaron en `results/outputs/dask/` y `results/kpis/dask/`, respectivamente.

```
GCS_PROCESSED_DASK = "gs://proyectobigdata2026/processed/dask/"
GCS_OUTPUTS_DASK = "gs://proyectobigdata2026/results/outputs/dask/"
GCS_KPIS_DASK = "gs://proyectobigdata2026/results/kpis/dask/"
```

Para evitar problemas de escritura directa hacia Google Cloud Storage desde pandas, los resultados pequeños se guardaron temporalmente en el entorno local y luego fueron subidos mediante `gcloud storage cp`. Esta estrategia fue utilizada para las tablas finales de CRUD y KPIs.

```
def save_pandas_csv_to_gcs(pdf, gcs_path, index=False):
    local_tmp = "/tmp/" + os.path.basename(gcs_path)
    pdf.to_csv(local_tmp, index=index)

    subprocess.run(
        f"gcloud storage cp {local_tmp} {gcs_path}",
        shell=True,
        check=True
    )
```

```
os.remove(local_tmp)
```

Con este flujo, Dask permitió procesar el dataset mediante particiones, generar una base limpia reutilizable y producir resultados analíticos almacenados en la nube. Esta implementación evidencia el uso de procesamiento paralelo, ejecución perezosa y materialización controlada de resultados dentro del pipeline Big Data.

5.2. Procesamiento con Apache Spark

Apache Spark fue utilizado como motor principal de procesamiento distribuido dentro del cluster de Dataproc. A diferencia de Dask y Modin, Spark se integra de forma nativa con la arquitectura del cluster, distribuyendo las tareas entre el nodo administrador y los nodos trabajadores. Por ello, resulta adecuado para ejecutar transformaciones, agregaciones, consultas CRUD y KPIs sobre el dataset almacenado en Google Cloud Storage.

En el proyecto, Spark se empleó para construir una base limpia comparable con las generadas por Dask y Modin, manteniendo los mismos criterios generales de limpieza: normalización de columnas, conversión de fechas, conversión del puntaje sanitario, tratamiento de valores inválidos, creación de variables auxiliares y construcción de una base válida para análisis.

Lectura del dataset desde Google Cloud Storage

El procesamiento con Spark inicia con la lectura del archivo original almacenado en la zona `raw/` del bucket del proyecto. A diferencia del entorno de prueba en Colab, donde se utilizaba una ruta local, en Dataproc el dataset se lee directamente desde Google Cloud Storage.

```
GCS_RAW_PATH = "gs://projectobigdata2026/raw/restaurant_inspections.csv"

df_raw = (
    spark.read
        .option("header", True)
        .option("inferSchema", False)
        .option("multiLine", False)
        .option("escape", "'")
        .csv(GCS_RAW_PATH)
)
```

La lectura se realiza sin inferencia automática de tipos para evitar inconsistencias entre particiones. Posteriormente, las columnas necesarias son convertidas explícitamente durante la etapa de limpieza.

Normalización y selección de columnas

Para mantener uniformidad con los notebooks de Dask y Modin, se normalizaron los nombres de las columnas a minúsculas y con guiones bajos. Esta convención permite trabajar con los mismos nombres en todas las herramientas y facilita la comparación de resultados.

```
def normalize_column_name(name):
    return (
        name.strip()
```

```

        .lower()
        .replace(" ", "_")
        .replace("-", "_")
    )

df = df_raw.toDF(*[normalize_column_name(c) for c in df_raw.columns])

```

Luego se seleccionaron las columnas necesarias para el análisis sanitario, incluyendo identificadores del restaurante, ubicación, fechas de inspección, puntaje, calificación, tipo de infracción, criticidad y variables geográficas.

Preparación y limpieza de datos

La limpieza con Spark siguió los mismos criterios generales aplicados en las otras herramientas. Primero se estandarizaron campos textuales mediante funciones como `trim`, `upper` y `lower`. Luego, las fechas fueron convertidas de forma flexible, considerando que el dataset puede contener fechas en formato ISO o en formato `MM/dd/yyyy`.

Variables auxiliares

Después de la limpieza base, se crearon variables auxiliares para facilitar el análisis posterior. Estas variables permiten construir consultas CRUD y KPIs de manera más directa.

- `year`: año de inspección.
- `month`: mes de inspección.
- `season`: clasificación estacional de la inspección.
- `is_critical`: indicador binario de infracción crítica.
- `has_violation`: indicador de existencia de infracción.
- `inspection_id`: identificador compuesto de inspección.
- `risk_level`: clasificación del riesgo sanitario según puntaje.
- `sanitary_status`: interpretación sanitaria basada en grado y criticidad.

```

df_clean = (
    df_clean
    .withColumn("year", year(col("inspection_date")))
    .withColumn("month", month(col("inspection_date")))
    .withColumn(
        "season",
        when(col("month").isin(6, 7, 8), "Verano")
        .when(col("month").isin(12, 1, 2), "Invierno")
        .otherwise("Otra temporada")
    )
    .withColumn(
        "is_critical",
        when(lower(col("critical_flag")) == "critical", 1).otherwise(0)
    )
)

```

```

    )
    .withColumn(
        "has_violation",
        when(col("violation_code").isNotNull(), 1).otherwise(0)
    )
)

```

La variable `inspection_id` es especialmente importante porque el dataset no debe interpretarse como una lista única de inspecciones. Una misma inspección puede aparecer en varias filas si registra múltiples infracciones. Por ello, se construyó un identificador compuesto a partir del restaurante, la fecha y el tipo de inspección.

```

df_clean = df_clean.withColumn(
    "inspection_id",
    concat_ws(
        "_",
        col("camis"),
        date_format(col("inspection_date"), "yyyy-MM-dd"),
        coalesce(col("inspection_type"), lit("unknown"))
    )
)

```

Corrección de valores inválidos en distrito

Al igual que en Dask, se identificaron valores inválidos en la variable `boro`, como 0, `Missing` o valores nulos. Para corregir estos casos, Spark construyó una tabla auxiliar que relaciona el código postal con el distrito a partir de registros válidos. Luego, esta tabla fue unida nuevamente con el dataset para generar la columna `boro_corrected`.

```

valid_boro_map = (
    df_clean
    .filter(
        col("zipcode").isNotNull() &
        col("boro").isNotNull() &
        (~col("boro").isin("0", "Missing", "MISSING", "missing"))
    )
    .groupBy("zipcode")
    .agg(first("boro", ignorenulls=True).alias("boro_from_zip"))
)

df_clean = (
    df_clean
    .join(valid_boro_map, on="zipcode", how="left")
    .withColumn(
        "boro_corrected",
        when(
            col("boro").isNull() |
            col("boro").isin("0", "Missing", "MISSING", "missing"),
            col("boro_from_zip")
        ).otherwise(col("boro"))
    )
    .withColumn(

```

```

        "boro_corrected",
        coalesce(col("boro_corrected"), lit("Sin datos"))
    )
    .drop("boro_from_zip")
)

```

Posteriormente, se construyó `df_analysis`, que contiene únicamente registros válidos para el análisis sanitario.

```

df_analysis = df_clean.filter(
    col("boro_corrected").isNotNull() &
    (col("boro_corrected") != "Sin datos") &
    col("score").isNotNull() &
    col("inspection_date").isNotNull()
)

```

Checkpoint en formato Parquet

Una vez construida la base limpia, Spark escribió el resultado en formato Parquet dentro de la carpeta `processed/spark/`. A diferencia del caso de Dask, donde fue necesario guardar por particiones de forma controlada debido a restricciones de memoria, Spark puede escribir el DataFrame directamente en Google Cloud Storage distribuyendo la tarea entre los ejecutores del cluster.

```

GCS_PROCESSED_SPARK = (
    "gs://proyectobigdata2026/processed/spark/"
    "restaurant_inspections_clean/"
)

(
    df_analysis
    .write
    .mode("overwrite")
    .parquet(GCS_PROCESSED_SPARK)
)

```

Luego, la base procesada fue leída nuevamente desde Parquet para ejecutar las consultas CRUD y los KPIs. Esto permite trabajar desde una base materializada y evita repetir la limpieza desde el archivo CSV original.

```

df_analysis = spark.read.parquet(GCS_PROCESSED_SPARK)

```

Consultas CRUD con Spark

Las consultas CRUD implementadas con Spark se resumen en la Tabla 14. Estas operaciones permiten demostrar creación de variables, lectura agregada, enriquecimiento del dataset y eliminación lógica de registros no válidos.

Cuadro 14: Consultas CRUD implementadas con Apache Spark

Código	Tipo	Descripción
SPARK-C1	Create	Creación de la variable <code>inspection_result</code> , que clasifica el resultado de la inspección según la calificación sanitaria.
SPARK-R1	Read	Agregación por distrito corregido, calculando inspecciones únicas, restaurantes únicos y puntaje promedio.
SPARK-U1	Update	Creación de la variable <code>priority_attention</code> , que clasifica registros según riesgo sanitario y criticidad.
SPARK-D1	Delete lógico	Construcción de un resumen de filtrado para cuantificar registros removidos entre la base original, limpia y analítica.
SPARK-R2	Read	Agregación por tipo de cocina, calculando inspecciones, restaurantes, puntaje promedio y tasa de infracciones críticas.

En estas consultas se utilizó `countDistinct(inspection_id)` para contar inspecciones únicas, evitando interpretar cada fila como una inspección independiente. Esta decisión es importante porque el dataset puede contener múltiples filas asociadas a una misma inspección.

Construcción de tabla a nivel de inspección

Antes del cálculo de KPIs, se creó una tabla intermedia a nivel de inspección. Esta tabla agrupa los registros por restaurante, fecha, tipo de inspección e identificador de inspección. Con ello, se evita que los KPIs temporales o de reincidencia se vean afectados por la repetición de filas derivada de múltiples infracciones en una misma inspección.

```
inspection_level = (
    df_analysis.groupBy(
        "camis",
        "dba",
        "boro_corrected",
        "cuisine_description",
        "inspection_date",
        "inspection_type",
        "inspection_id",
        "year",
        "month",
        "season"
    )
    .agg(
        round(avg("score"), 2).alias("inspection_score"),
        spark_sum("is_critical").alias("critical_violations"),
    )
)
```

```

    spark_sum("has_violation").alias("total_violations")
  )
)

```

Esta tabla fue utilizada como base para los KPIs principales de Spark, especialmente aquellos que requieren comparación temporal, agregación por zona o análisis histórico de desempeño.

KPIs calculados con Spark

Spark fue utilizado para calcular indicadores que requieren operaciones distribuidas más complejas, como funciones de ventana, uniones, agregaciones por zona y análisis temporal. Para evitar contar cada fila como una inspección independiente, previamente se construyó una tabla a nivel de inspección mediante la variable `inspection_id`. Esta decisión permite reducir el sesgo generado por inspecciones con múltiples infracciones registradas.

Cuadro 15: KPIs calculados con Apache Spark

KPI	Nombre	Propósito
KPI 1	Índice de efectividad de reinspección	Evaluar si las reinspecciones reducen el puntaje sanitario respecto a una inspección previa.
KPI 2	Índice de deterioro estacional	Comparar el puntaje promedio en verano frente al invierno por distrito y tipo de cocina.
KPI 5	Índice de riesgo sanitario por zona geográfica	Priorizar distritos combinando puntaje promedio, carga crítica y reincidencia.
KPI 6	Índice de excelencia sostenida	Medir la proporción de restaurantes que mantienen cero infracciones críticas en los últimos dos años.

KPI 1: Índice de efectividad de reinspección **Definición.** Este KPI mide si las reinspecciones sanitarias logran reducir el puntaje sanitario de los restaurantes respecto a su inspección anterior. Como en este dataset un menor puntaje representa un mejor desempeño sanitario, una reducción del `score` indica mejora.

Fórmula.

$$\text{Efectividad de reinspección} = \frac{\text{Score anterior} - \text{Score de reinspección}}{\text{Score anterior}} \times 100$$

Se analizaron 18 938 reinspecciones y se obtuvo una efectividad promedio de 30.1 %. Esto indica que, en promedio, las reinspecciones muestran una reducción del puntaje sanitario respecto a la inspección anterior.

Cuadro 16: Resumen del KPI 1: efectividad de reinspección

Reinspecciones analizadas	Efectividad promedio
18 938	30.1 %

Cuadro 17: Ejemplos de reinspecciones analizadas con Spark

CAMIS	Restaurante	Distrito	Score ant.	Score nuevo	Efect.	Resultado
30075445	MORRIS PARK BAKE SHOP	Bronx	38.0	12.0	68.42 %	Mejóro
30191841	D.J. REYNOLDS	Manhattan	24.0	10.0	58.33 %	Mejóro
40363298	CAFE METRO	Manhattan	26.0	2.0	92.31 %	Mejóro
40363427	BAGELS N BUNS	Staten Island	37.0	10.0	72.97 %	Mejóro
40363834	CARVEL	Staten Island	18.0	3.0	83.33 %	Mejóro
40364347	METROPOLITAN CLUB	Manhattan	10.0	13.0	-30.0 %	Empeoró
40364467	NYAC MAIN DINING ROOM	Manhattan	25.0	4.0	84.0 %	Mejóro
40365525	DONUT PUB	Manhattan	60.0	11.0	81.67 %	Mejóro

El resultado promedio positivo sugiere que las reinspecciones suelen estar asociadas con una mejora sanitaria. Sin embargo, existen casos donde el indicador es negativo, como *METROPOLITAN CLUB*, donde el puntaje aumentó de 10 a 13, indicando un deterioro relativo. Este KPI permite evaluar si las acciones posteriores a una inspección inicial producen mejoras observables.

KPI 2: Índice de deterioro estacional **Definición.** Este KPI compara el puntaje promedio de inspección en verano frente al invierno. El análisis se realiza por distrito y tipo de cocina para identificar segmentos donde el desempeño sanitario empeora durante los meses de verano.

Fórmula.

$$\text{Deterioro estacional} = \frac{\text{Score promedio verano} - \text{Score promedio invierno}}{\text{Score promedio invierno}} \times 100$$

La Tabla 18 muestra los mayores valores del índice de deterioro estacional.

Cuadro 18: Top 10 segmentos con mayor deterioro estacional

Distrito	Tipo de cocina	Score verano	Score invierno	Deterioro
Staten Island	Caribbean	10.5	2.0	425.0 %
Manhattan	Basque	41.0	9.0	355.56 %
Queens	German	21.5	5.6	283.93 %
Bronx	Moroccan	38.0	10.0	280.0 %
Brooklyn	Cajun	42.0	12.0	250.0 %
Queens	Not Listed/Not Applicable	10.0	3.0	233.33 %
Staten Island	Jewish/Kosher	58.0	17.75	226.76 %
Staten Island	Thai	32.33	10.0	223.3 %
Brooklyn	Not Listed/Not Applicable	24.0	8.0	200.0 %
Bronx	Vegan	20.33	8.0	154.12 %

Los resultados muestran que algunos segmentos presentan un puntaje promedio considerablemente mayor en verano que en invierno, lo que indica deterioro sanitario estacional. Sin embargo, varios de los valores más altos corresponden a grupos con pocas inspecciones, por lo que deben interpretarse con cautela. Para un análisis más robusto, podría aplicarse un umbral mínimo de inspecciones por temporada antes de considerar un segmento como prioritario.

KPI 5: Índice de riesgo sanitario por zona geográfica **Definición.** Este KPI combina tres dimensiones por distrito: puntaje promedio, carga de infracciones críticas y reincidencia de restaurantes. Su objetivo es identificar qué zonas concentran mayor riesgo sanitario relativo.

Fórmula.

Índice de riesgo sanitario = Score promedio + Carga crítica + Tasa de reincidencia

Donde:

$$\text{Carga crítica} = \frac{\text{Infracciones críticas}}{\text{Total de inspecciones}} \times 100$$

$$\text{Tasa de reincidencia} = \frac{\text{Restaurantes reincidentes}}{\text{Total de restaurantes}} \times 100$$

En este caso, la carga crítica puede superar el 100 %, ya que una misma inspección puede contener más de una infracción crítica. Por tanto, no debe interpretarse como porcentaje de inspecciones críticas, sino como cantidad de infracciones críticas por cada 100 inspecciones.

Cuadro 19: Índice de riesgo sanitario por distrito

Distrito	Insp.	Rest.	Score prom.	Carga crítica	Reincid.	Índice
Queens	19 508	6 288	19.06	201.54	97.92	318.52
Bronx	7 691	2 412	17.46	185.07	97.26	299.79
Brooklyn	21 760	7 029	17.82	183.75	97.23	298.80
Staten Island	3 001	1 006	16.95	182.21	97.81	296.97
Manhattan	31 695	10 834	17.09	179.70	96.87	293.66

Queens presenta el mayor índice de riesgo sanitario, con 318.52, debido a su mayor puntaje promedio, mayor carga crítica y alta reincidencia. Manhattan, aunque concentra el mayor número de inspecciones y restaurantes, presenta el índice más bajo entre los cinco distritos. Este KPI permite priorizar zonas donde el riesgo sanitario agregado es más alto, aunque debe interpretarse como un índice compuesto y no como una métrica absoluta de incumplimiento.

KPI 6: Índice de excelencia sostenida **Definición.** Este KPI mide la proporción de restaurantes que mantuvieron cero infracciones críticas durante los últimos dos años. Para dar mayor confiabilidad al indicador, solo se consideran restaurantes con al menos tres inspecciones en dicho periodo.

$$\text{Índice de excelencia sostenida} = \frac{\text{Restaurantes sin infracciones críticas}}{\text{Restaurantes con al menos 3 inspecciones}} \times 100$$

Resultado general.

Cuadro 20: Resumen general del índice de excelencia sostenida

Restaurantes evaluados	Restaurantes excelentes	Índice
5 193	16	0.31 %

Resultado por distrito.

Cuadro 21: Índice de excelencia sostenida por distrito

Distrito	Rest. evaluados	Rest. excelentes	Índice
Brooklyn	1 352	7	0.52 %
Manhattan	1 785	6	0.34 %
Queens	1 336	3	0.22 %
Bronx	588	0	0.00 %
Staten Island	132	0	0.00 %

El índice general de excelencia sostenida es bajo: solo 16 de 5 193 restaurantes evaluados mantuvieron cero infracciones críticas durante el periodo considerado, equivalente al 0.31 %. Brooklyn presenta el mayor valor relativo, con 0.52 %, seguido por Manhattan con 0.34 %. Bronx y Staten Island no registran restaurantes que cumplan el criterio de excelencia sostenida. Este KPI muestra que mantener un desempeño sanitario sin infracciones críticas durante varias inspecciones es poco frecuente.

Almacenamiento de resultados

Los resultados generados con Spark fueron almacenados nuevamente en Google Cloud Storage. Los datos procesados se guardaron en la carpeta `processed/spark/`, mientras que las salidas de consultas CRUD y KPIs se almacenaron en `results/outputs/spark/` y `results/kpis/spark/`, respectivamente.

Para las tablas finales se utilizó escritura en formato CSV con encabezados, consolidando la salida en un archivo por consulta o KPI mediante `coalesce(1)` cuando el resultado era pequeño.

```
def write_csv_spark(df_spark, path):
    (
        df_spark
        .coalesce(1)
        .write
        .mode("overwrite")
        .option("header", True)
        .csv(path)
    )
```

Con este flujo, Spark permitió ejecutar procesamiento distribuido directamente sobre Dataproc, generar una base limpia en formato Parquet, construir consultas CRUD y calcular KPIs que requieren agregaciones, ventanas temporales y análisis histórico. Esta implementación evidencia el uso de Spark como motor distribuido dentro del pipeline Big Data en GCP.

5.3. Procesamiento con Modin

Modin fue utilizado como una herramienta de procesamiento paralelo con una interfaz similar a pandas. Su principal ventaja es que permite escribir código con una sintaxis familiar para análisis tabular, mientras distribuye internamente ciertas operaciones mediante un motor de ejecución paralelo, en este caso Ray.

A diferencia de Spark, que opera como un motor distribuido basado en driver y executors, Modin se orienta a facilitar la transición desde pandas hacia un entorno paralelo. Por ello, en este proyecto se empleó para realizar limpieza, transformación, consultas CRUD y KPIs manteniendo una estructura de código cercana a pandas, pero integrada al flujo de trabajo en Google Cloud Platform.

Configuración del entorno Modin

Para ejecutar Modin en el entorno Jupyter de Dataproc, se utilizó el backend Ray. Esta configuración permite paralelizar operaciones compatibles con pandas dentro del entorno disponible en el cluster.

```
import os
import ray
import modin.pandas as pd

os.environ["MODIN_ENGINE"] = "ray"
os.environ["MODIN_CPUS"] = "2"

ray.init(
    ignore_reinit_error=True,
    num_cpus=2,
    include_dashboard=False
)
```

Esta configuración limita el uso de CPU para mantener controlado el consumo de recursos en el entorno interactivo. Modin fue utilizado principalmente para operaciones tabulares, agregaciones, filtros, imputación de valores y generación de indicadores.

Lectura del dataset desde Google Cloud Storage

En la versión inicial del notebook, el dataset era leído desde Google Drive. Para integrar el procesamiento al pipeline en GCP, se reemplazó esa lógica por una lectura desde Google Cloud Storage. Debido a que Modin trabaja con una API similar a pandas, se utilizó una estrategia segura: copiar el archivo desde GCS al almacenamiento temporal del nodo y luego cargarlo con Modin.

```
GCS_RAW_PATH = "gs://proyectobigdata2026/raw/restaurant_inspections.csv"
LOCAL_RAW_PATH = "/tmp/restaurant_inspections.csv"

subprocess.run(
    f"gcloud storage cp {GCS_RAW_PATH} {LOCAL_RAW_PATH}",
    shell=True,
    check=True
)

df_raw = pd.read_csv(
    LOCAL_RAW_PATH,
    dtype=dtype_config
)
```

Este enfoque evita depender de rutas locales de Colab o Google Drive y permite que el procesamiento se ejecute dentro del cluster de Dataproc, manteniendo el dataset original en la capa `raw/` del Data Lake.

Normalización y perfilamiento inicial

Al igual que en Dask y Spark, se normalizaron los nombres de columnas para utilizar una convención uniforme en todo el proyecto. Las columnas fueron convertidas a minúsculas y los espacios fueron reemplazados por guiones bajos.

```
def normalize_column_name(name):
    return (
        name.strip()
        .lower()
        .replace(" ", "_")
        .replace("-", "_")
    )

df_raw.columns = [normalize_column_name(c) for c in df_raw.columns]
```

Luego, se realizó un perfilamiento inicial para revisar valores nulos y distribuciones relevantes, como `boro`, `grade`, `score`, `critical_flag` y `violation_code`. Esta revisión permitió identificar registros incompletos, fechas placeholder y categorías inválidas.

Preparación y limpieza de datos

La limpieza con Modin siguió los mismos criterios generales aplicados con Dask y Spark. Primero se estandarizaron los campos textuales, eliminando espacios innecesarios y convirtiendo algunas variables a mayúsculas para evitar duplicidades en agrupaciones. Así mismo, se corrigieron los valores inválidos en boro al igual que los casos anteriores. Además, se aplicó una imputación de calificación sanitaria basada en el puntaje. A diferencia del notebook inicial, no se sobrescribió la columna `grade`; se creó una nueva variable llamada `grade_imputed` para conservar el valor original y, al mismo tiempo, recuperar información cuando la calificación estaba ausente.

```
def grade_from_score(score):
    if pd.isna(score):
        return None
    if score <= 13:
        return "A"
    if score <= 27:
        return "B"
    return "C"

df_clean["grade_imputed"] = df_clean["grade"]

missing_grade = df_clean["grade_imputed"].isna()

df_clean.loc[missing_grade, "grade_imputed"] = df_clean.loc[
    missing_grade,
    "score"
].apply(grade_from_score)
```

Variables específicas para KPIs en Modin

Adicionalmente, se crearon variables específicas para los KPIs calculados con Modin. Entre ellas se encuentran `has_inspection`, `closure_flag` y `severity_score`. Esta última asigna un peso mayor a las infracciones críticas, permitiendo medir la severidad promedio de las violaciones.

```
df_clean["has_inspection"] = df_clean["inspection_date"].notna()

df_clean["closure_flag"] = (
    df_clean["action"]
    .fillna("")
    .str.lower()
    .str.contains("closed", na=False)
)

critical_norm = df_clean["critical_flag"].fillna("").str.lower()

df_clean["severity_score"] = np.select(
    [
        critical_norm.eq("critical"),
        critical_norm.eq("not critical")
    ],
```



```
[
    2,
    1
],
default=0
)
```

Base final y checkpoint en Parquet

La base final de análisis se construyó filtrando registros con distrito corregido válido, puntaje disponible, fecha de inspección y acción registrada. Con ello se obtiene una base comparable con las generadas por Dask y Spark.

```
df_analysis = df_clean[
    df_clean["boro_corrected"].notna() &
    (df_clean["boro_corrected"] != "Sin datos") &
    df_clean["score"].notna() &
    df_clean["inspection_date"].notna() &
    df_clean["action"].notna()
].copy()
```

Posteriormente, esta base se guardó como checkpoint en formato Parquet y se subió a Google Cloud Storage.

```
df_analysis.to_parquet(
    "/tmp/modin_restaurant_inspections_clean.parquet",
    engine="pyarrow",
    index=False
)

subprocess.run(
    "gcloud storage cp "
    "/tmp/modin_restaurant_inspections_clean.parquet "
    "gs://proyectobigdata2026/processed/modin/"
    "restaurant_inspections_clean.parquet",
    shell=True,
    check=True
)
```

Este checkpoint permite conservar una versión limpia de los datos procesados con Modin y reutilizarla para consultas CRUD y KPIs sin repetir la limpieza desde el archivo CSV original.

Consultas CRUD con Modin

Las operaciones CRUD implementadas con Modin se resumen en la Tabla 22. Estas consultas permiten demostrar creación de variables, lectura de subconjuntos, agregación, actualización mediante imputación y eliminación lógica de registros no válidos.

Cuadro 22: Consultas CRUD implementadas con Modin

Código	Tipo	Descripción
MODIN-C1	Create	Creación de la variable <code>has_inspection</code> , que indica si el registro cuenta con fecha de inspección válida.
MODIN-R1	Read	Consulta de restaurantes de alto riesgo, considerando registros con <code>score</code> mayor a 30.
MODIN-R2	Read	Agregación de restaurantes únicos por tipo de cocina mediante <code>groupby</code> y <code>nunique</code> .
MODIN-U1	Update	Imputación de la variable <code>grade_imputed</code> a partir del valor de <code>score</code> , sin sobrescribir la calificación original.
MODIN-D1	Delete lógico	Construcción de una base analítica válida al filtrar registros sin distrito corregido, puntaje, fecha o acción registrada.

En esta versión se descartó la inserción de un registro ficticio como operación *Create*, ya que podía alterar artificialmente el dataset. En su lugar, se utilizaron variables derivadas reales que enriquecen el análisis sin modificar la naturaleza de los datos originales.

KPIs calculados con Modin

Modin fue utilizado para calcular indicadores orientados a severidad sanitaria, reincidencia geográfica y presión de inspección. Estos KPIs aprovechan operaciones tabulares similares a pandas, como filtros, agrupaciones, conteos únicos, uniones y cálculos de proporciones, pero ejecutadas mediante el backend paralelo de Modin con Ray.

Cuadro 23: KPIs calculados con Modin

KPI	Nombre	Propósito
KPI 8	Índice de severidad de violación	Medir la severidad promedio de las infracciones asignando mayor peso a violaciones críticas.
KPI 9	Reincidencia geográfica	Identificar zonas donde una mayor proporción de restaurantes presenta infracciones críticas recurrentes.
KPI 10	Índice de presión de inspección	Medir la frecuencia de inspecciones por restaurante en cada distrito.

KPI 8: Índice de severidad de violación **Definición.** Este KPI mide la severidad promedio de las infracciones sanitarias. Para ello, se asigna un peso según la criticidad

del registro: 2 para infracciones críticas, 1 para infracciones no críticas y 0 para casos no aplicables o sin criticidad definida.

Fórmula.

$$\text{Índice de severidad} = \frac{\sum \text{severity_score}}{N}$$

Donde N representa el total de registros considerados en la base de análisis.

Resultado general.

Cuadro 24: Índice global de severidad de violación

Métrica	Valor
Índice global de severidad	1.5524

Resultado por tipo de cocina.

Cuadro 25: Top 10 tipos de cocina con mayor severidad promedio

Tipo de cocina	Severidad promedio
Czech	1.6875
Bangladeshi	1.6215
Creole	1.6096
Chinese/Japanese	1.5985
Pakistani	1.5982
Continental	1.5967
African	1.5937
Indian	1.5900
Polish	1.5896
Armenian	1.5867

Resultado por distrito.

Cuadro 26: Índice de severidad promedio por distrito

Distrito	Severidad promedio
Staten Island	1.5726
Queens	1.5616
Brooklyn	1.5527
Bronx	1.5479
Manhattan	1.5453

El índice global de severidad fue 1.5524, lo que indica que, en promedio, los registros tienden a ubicarse entre infracciones no críticas y críticas, con mayor peso hacia infracciones de severidad relevante. Por tipo de cocina, categorías como *Czech*, *Bangladeshi*, *Creole*, *Chinese/Japanese* y *Pakistani* presentan los valores más altos. Por distrito, Staten Island

y Queens muestran las mayores severidades promedio. Este KPI permite identificar segmentos donde las infracciones tienden a ser más graves, aunque debe interpretarse junto con el volumen de inspecciones de cada grupo.

KPI 9: Reincidencia geográfica **Definición.** Este KPI mide la proporción de restaurantes reincidentes dentro de cada zona geográfica. Un restaurante se considera reincidente si registra infracciones críticas en más de una fecha de inspección. El análisis se realiza a nivel de código postal para obtener una visión más granular que el distrito.

Fórmula.

$$\text{Tasa de reincidencia geográfica} = \frac{\text{Restaurantes reincidentes}}{\text{Total de restaurantes de la zona}} \times 100$$

Para evitar resultados poco representativos, se consideran únicamente códigos postales con al menos 10 restaurantes.

Cuadro 27: Top 10 códigos postales con mayor reincidencia geográfica

ZIP Code	Restaurantes	Reincidentes	Tasa
11411	22	21	95.45 %
10039	19	18	94.74 %
11228	62	58	93.55 %
10470	41	37	90.24 %
10118	10	9	90.00 %
10466	85	76	89.41 %
11356	62	55	88.71 %
10310	61	54	88.52 %
10453	94	83	88.30 %
10474	32	28	87.50 %

Los resultados muestran zonas con tasas muy altas de reincidencia crítica. El código postal 11411 presenta la mayor tasa, con 21 restaurantes reincidentes de 22 evaluados, equivalente al 95.45 %. También destacan 10039 y 11228, con tasas superiores al 93 %. Este KPI permite identificar zonas donde los problemas sanitarios no parecen ser eventos aislados, sino comportamientos recurrentes dentro de un conjunto importante de establecimientos.

KPI 10: Índice de presión de inspección **Definición.** Este KPI mide la frecuencia con la que los restaurantes de un distrito son inspeccionados. Se calcula dividiendo el total de inspecciones únicas entre la cantidad de restaurantes únicos del distrito.

Fórmula.

$$\text{Presión de inspección} = \frac{\text{Total de inspecciones}}{\text{Restaurantes únicos}}$$

Un valor mayor indica que, en promedio, los restaurantes de ese distrito reciben más inspecciones.

Resultado.

Cuadro 28: Índice de presión de inspección por distrito

Distrito	Inspecciones	Restaurantes	Presión
Bronx	7 691	2 412	3.19
Brooklyn	21 760	7 029	3.10
Queens	19 508	6 288	3.10
Staten Island	3 001	1 006	2.98
Manhattan	31 695	10 834	2.93

Bronx presenta la mayor presión de inspección, con 3.19 inspecciones por restaurante. Brooklyn y Queens presentan valores similares, ambos con 3.10. Manhattan, aunque tiene el mayor número absoluto de inspecciones y restaurantes, presenta la menor presión relativa, con 2.93 inspecciones por restaurante. Este indicador permite comparar la intensidad de fiscalización entre distritos y puede ayudar a identificar zonas con mayor seguimiento sanitario.

En conjunto, los KPIs calculados con Modin complementan los resultados obtenidos con Dask y Spark. El índice de severidad permite evaluar la gravedad promedio de las infracciones; la reincidencia geográfica permite ubicar zonas con problemas críticos repetidos; y la presión de inspección permite comparar la intensidad de fiscalización entre distritos.

Almacenamiento de resultados

Los resultados generados con Modin fueron almacenados nuevamente en Google Cloud Storage. El checkpoint limpio se guardó en `processed/modin/`, mientras que las consultas CRUD y KPIs se almacenaron en `results/outputs/modin/` y `results/kpis/modin/`.

Para guardar resultados pequeños, se utilizó una función que escribe temporalmente el archivo en el nodo de Jupyter y luego lo sube al bucket mediante `gcloud storage cp`.

```
def save_modin_csv_to_gcs(df_result, gcs_path, index=False):
    local_tmp = "/tmp/" + os.path.basename(gcs_path)

    df_result.to_csv(local_tmp, index=index)

    subprocess.run(
        f"gcloud storage cp {local_tmp} {gcs_path}",
        shell=True,
        check=True
    )

    os.remove(local_tmp)
```

6. Conclusiones

El proyecto permitió cumplir el objetivo general de diseñar e implementar un pipeline Big Data en la nube, integrando Google Cloud Storage como capa de almacenamiento,

Dataprocc como entorno de procesamiento distribuido y Dask, Modin y Apache Spark como herramientas de análisis. Esta arquitectura permitió organizar el flujo completo desde la ingesta del dataset original hasta la generación de datos procesados, consultas CRUD y KPIs sanitarios.

Desde el punto de vista técnico, Google Cloud Storage funcionó como un Data Lake ordenado, separando los datos crudos, procesados y resultados analíticos. Asimismo, Dataprocc permitió trabajar sobre una arquitectura distribuida real, con un nodo administrador y nodos trabajadores. Dask facilitó el procesamiento por particiones y la ejecución perezosa; Modin permitió adaptar operaciones similares a pandas en un entorno paralelo; y Spark fue especialmente útil para agregaciones, análisis temporal y procesamiento distribuido a mayor escala.

En cuanto al análisis sanitario, los KPIs construidos permitieron identificar patrones relevantes en las inspecciones de restaurantes de Nueva York. Los resultados muestran que el riesgo sanitario no se distribuye de manera uniforme entre distritos, tipos de cocina y establecimientos. Además, se identificaron casos de reincidencia, presencia de infracciones críticas, diferencias en severidad y establecimientos con tiempos prolongados de subsanación.

También se evidenció que las reinspecciones pueden contribuir a mejorar el desempeño sanitario, aunque no garantizan una corrección uniforme en todos los restaurantes. Por ello, el análisis histórico resulta más útil que una evaluación basada en una sola inspección, ya que permite distinguir mejoras puntuales de comportamientos sanitarios sostenidos.

Finalmente, el proyecto demuestra que una solución Big Data no solo consiste en procesar grandes volúmenes de datos, sino en transformar esa información en indicadores útiles para la toma de decisiones. La combinación de GCP, Dask, Modin y Spark permitió construir un pipeline funcional, reproducible y orientado al análisis sanitario, cumpliendo con los requerimientos de almacenamiento en la nube, procesamiento distribuido, operaciones CRUD y generación de KPIs.

7. Referencias

New York City Department of Health and Mental Hygiene. (s. f.). *Food establishment inspections*. NYC Health. <https://www.nyc.gov/site/doh/services/restaurant-grades.page>

NYC OpenData. (s. f.). *DOHMH New York City Restaurant Inspection Results* [Data set]. Recuperado el 5 de mayo de 2026, de <https://data.cityofnewyork.us/Health/DOHMH-New-York-City-Restaurant-Inspection-Results/43nn-pn8j>